

Week10_예습과제_백재은

파이썬 머신러닝 완벽 가이드 Ch.6 차원축소

01. 차원 축소 개요

a. 차원축소

- 매우 많은 피처로 구성된 다차원 데이터 세트의 차원을 축소해 새로운 차원의 데이터 세트를 생성하는 것
- 차원 up, 데이터 포인트 간 거리 up, 희소한 구조 => General, 예측 신뢰도 down
- 차원 축소를 통해 직관 해석, 처리 능력 down 가능

b. 차원 축소의 구분

- 피처 선택
 - 특정 피처에 종속성이 강한 불필요 피처 아예 제거, 주요 피처만 선택
- 피처 추출
 - 기존 피처를 저차원의 중요 피처로 압축해서 추출 (다른 값이 됨)
 - 다른 공간으로 매칭하여 추출

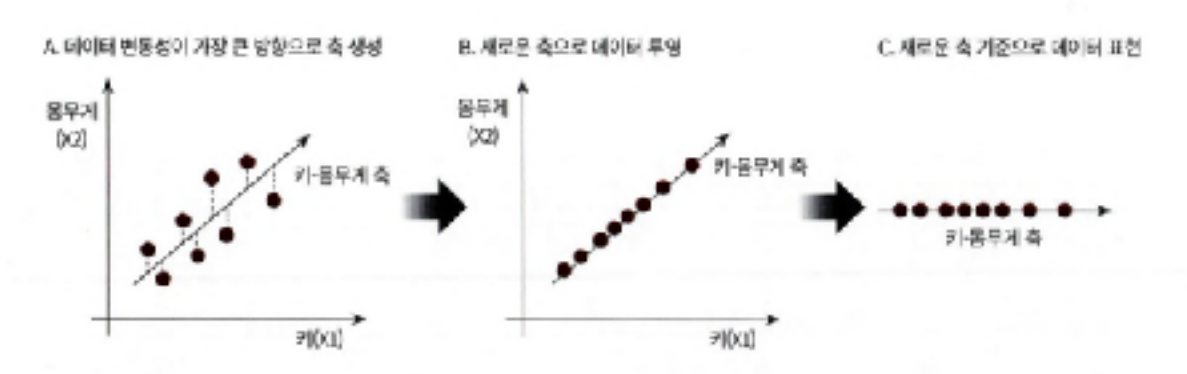
c. 차원 축소 알고리즘

- 많은 차원을 가진 이미지, 텍스트로부터 차원 축소를 통해 잠재적인 의미를 찾아주는 알고리즘
- 예시: PCA, SVD, NMF
- 함축적 형태의 이미지 변환과 압축 수행 => 과적합 영향력 작아짐
- 텍스트 문서의 숨겨진 의미 추출 -> 시맨틱 의미나 토픽 잠재 요소 간주, 파악

02. Principal Component Analysis

a. 개요

- 여러 변수 간의 상관관계를 이용해 이를 대표하는 주성분을 추출해 차원을 축소하는 기법
- 기존 데이터의 정보 유실이 최소화되는 것이 당연
- 가장 높은 분산을 가지는 데이터의 축을 찾아 이 축으로 차원 축소 = 주성분



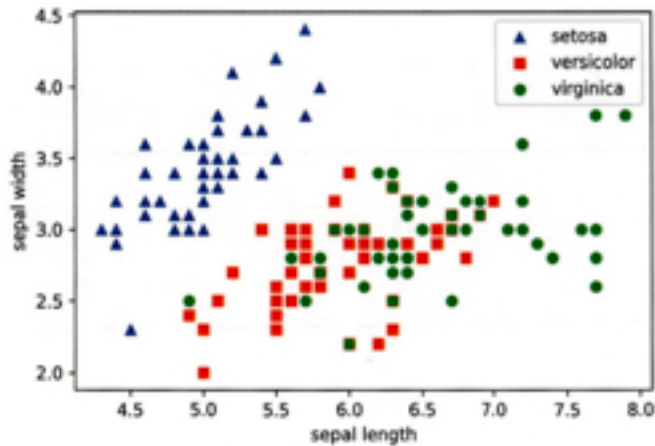
- 키와 몸무게, 2개 피처를 가진 데이터 세트를 가정했을 때 한 개의 주성분을 가진 데이터 세트로 차원 축소 => 데이터 변동성이 가장 큰 방향으로 축 생성 => new 축으로 데이터 투영
- 첫 번째 벡터 축 = 가장 큰 데이터 변동성 기반
- 두 번째 벡터 축 = 위 벡터 축의 직교 벡터 ... 반복 => 벡터 축의 개수만큼의 차원으로 원본 데이터 차원 축소
- 선형 대수 관점: 입력 데이터의 공분산 행렬을 고유값 분해한 후, 해당 고유벡터에 입력 데이터를 선형 변환하는 것
- 고유벡터: PCA의 주성분 벡터, 입력 데이터의 분산이 큰 방향 / 고윳값: 고유벡터의 크기, 입력 데이터의 분산
- 선형 변환: 특정 벡터에 행렬 A를 곱해 새로운 벡터로 변환하는 것
- 분산: 한 개의 특정한 변수의 데이터 변동 / 공분산: 두 변수 간의 변동 / 공분산 행렬: 여러 변수와 관련된 공분산을 포함하는 정방행렬 (정방행렬, 대칭행렬 성질)
- 고유벡터: 행렬 A를 곱하더라도 방향이 변하지 않고 그 크기만 변하는 벡터

$$C = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^t \\ \cdots \\ e_n^t \end{bmatrix}$$

- 공분산 C의 분해: 고유벡터 직교 행렬 * 고유값 정방 행렬 * 고유벡터 직교 행렬의 전치 행렬
- 해당 고유 벡터를 이용하여 입력 데이터를 선형 변환하는 방식, 알고리즘 => PCA

b. 붓꽃 데이터 세트

1) PCA 수행



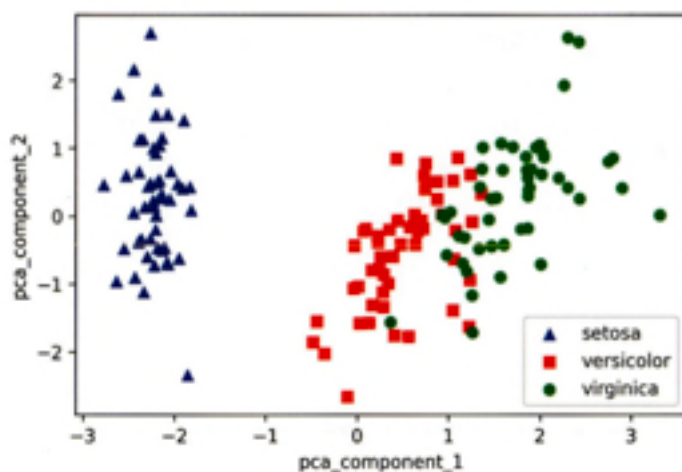
~sepal length, sepal width를 X축, Y축
으로 하여 품종 데이터 분포를 나타냄

<Output> (150, 2)

- 붓꽃 데이터 세트에 평균이 0, 분산이 1인 표준 정규 분포로 속성값 변환, 스케일링
- 4차원 붓꽃 데이터를 2차원 PCA 데이터로 변환, n_components (차원 수) = 2로 입력 받아 변환 수행

	pca_component_1	pca_component_2	target
0	-2.264703	0.480027	0
1	-2.080961	-0.674134	0
2	-2.364229	-0.341908	0

~변환된 PCA 데이터 세트 150x2 넘파이 행렬로
df 출력



~pca_component_1 속성 X축,
pca_component_2 속성 Y축으로 두어 2
차원상 시각화

~pca_component_1 => 원본 데이터 변
동성 잘 반영함

2) 개별 PCA 컴포넌트별로 원본 데이터의 변동성 반영 정도

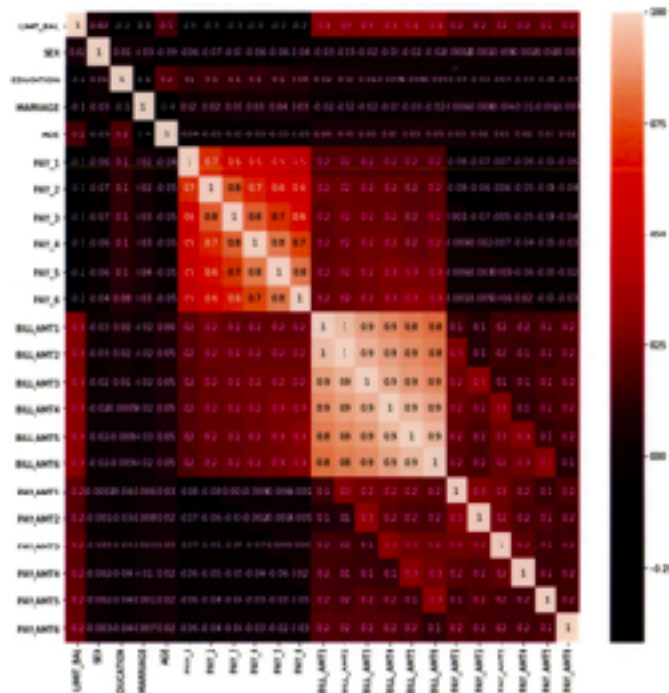
- experienced_variance_ratio_ 속성 = 전체 변동성에서 개별 PCA 컴포넌트별로 차지하는 변동성 비율 제공

3) 분류 적용, 결과 비교

- 원본 붓꽃 데이터 세트와 변환 데이터 세트에 randomforest로 측정한 후, corss_val_score()로 교차 검증하여 정확도 결과 비교
- 원본 평균: 0.96 / 변환 평균: 0.88 -> 변환 후에도 특성을 상당 부분 유지하고 있음

c. 신용카드 고객 데이터 세트

-> 더 많은 피처를 가진 데이터 세트를 적은 PCA 컴포넌트 기반으로 변환한 뒤 예측 영향도의 변화를 비교해보고자 함



- corr() 을 통해 속성 간 상관도를 시본 히트맵으로 시각화

- 높은 상관도를 가진 속성들(BILL_AMT1 ~ BILL_AMT6)을 소수의 PCA를 통해 변동성 수용 가능

- BILL_AMT1 ~ BILL_AMT6을 2개의 컴포넌트로 변환한 뒤 개별 컴포넌트의 변동성 알아보기 => 0.9055... => 첫 번째 PCA 축으로 90%의 변동성 수용, 상관도 매우 높음

- 원본과 변환한 데이터 세트의 분류 예측 결과 상호 비교 by random forest

=> 원본 평균 정확도: 0.8170 / PCA 변환 정확도: 0.7973 ... 뛰어난 압축 능력

03. Linear Discriminant Analysis

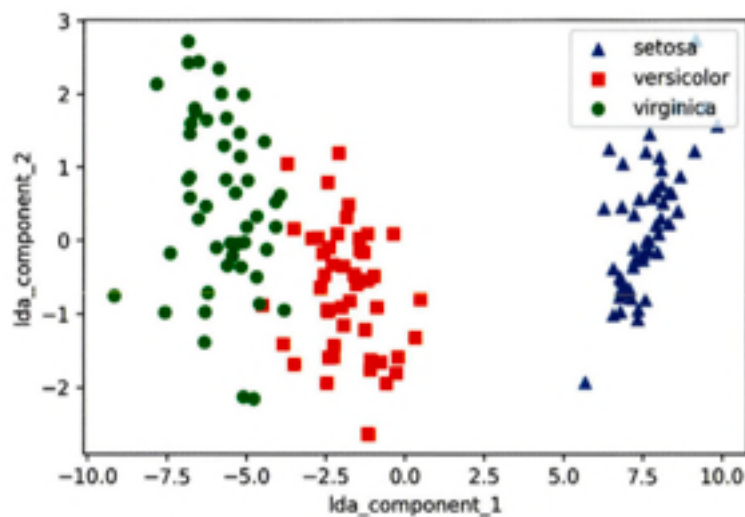
a. 개요

- LDA, 선형 판별 분석법: 개별 클래스를 분별할 수 있는 기준을 최대한 유지하면서 차원 축소, 입력 데이터 세트를 저차원 공간에 투영해 차원을 축소하는 기법, 입력 데이터의 결정 값 클래스를 최대한으로 분리할 수 있는 축 찾기
- 클래스 간 분산과 클래스 내부 분산의 비율을 최대화하는 방식으로 차원 축소
- 클래스 간 분산과 클래스 내부 분산 행렬 생성 -> 해당 행렬 기반 고유벡터 구한 후 입력 데이터 투영

$$S_W^T S_B = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^T \\ \cdots \\ e_n^T \end{bmatrix}$$

b. 붓꽃 데이터 세트에 LDA 적용

- 1) 붓꽃 데이터 세트 로드, 표준 정규 분포로 스케일링
- 2) LDA 변환, 클래스 결정 값 입력
- 3) 입력 데이터 값 -> 2차원 평면에 품종별로 표현



-PCA와 유사

04. Singular Value Decomposition

a. SVD 개요

- 특이값 분해, $m \times n$ 크기의 행렬 A 를 다음과 같이 분해
- U 와 V : 특이벡터, 서로 직교하는 성질 / 시그마: 대각행렬 / 0이 아닌 값: 행렬 A 의 특이값

$$A = U \Sigma V^T$$

- A 의 차원이 $m \times n$ 일 때, U 의 차원이 $m \times m$, 시그마의 차원이 $m \times n$, V^T 의 차원이 $n \times n$ 으로 분해
- Truncate SVD: 시그마의 대각원소 중 상위 몇 개만 추출해서 이에 대응하는 U 와 V 의 원소도 함께 제거하여 더욱 차원을 줄인 형태로 분해하는 것
- 이미지 압축을 통한 패턴 인식, 텍스트의 토픽 모델링 기법, 문서 유사도 및 클러스터링 활용

b. 예제

- 넘파이 SVD 모듈 `bumpy.linalg.svd` field -> 랜덤 4 x 4 행렬 생성
- 해당 a 행렬에 SVD 적용, U , Σ , V^T 도출

```
(4, 4) (4, ) (4, 4)
U matrix:
[[-0.079 -0.318  0.867  0.376]
 [ 0.383  0.787  0.12  0.469]
 [ 0.656  0.022  0.357 -0.664]
 [ 0.645 -0.529 -0.328  0.444]]
Sigma Value:
[3.423 2.023 0.463 0.079]
```

```
V transpose matrix:
[[ 0.041  0.224  0.786 -0.574]
 [-0.2   0.562  0.37  0.712]
 [-0.778  0.395 -0.333 -0.357]
 [-0.593 -0.692  0.366  0.189]]
```

-> U 행렬 4 x 4

-> V^T 행렬 4 x 4

-> Σ 1차원 행렬 (4,)

- 복원 (내적 수행)

```
[[-0.212 -0.285 -0.574 -0.44 ]
 [-0.33  1.184  1.615  0.367]
 [-0.014  0.63  1.71  -1.327]
 [ 0.402 -0.191  1.404 -1.969]]
```

-> 원본 a와 동일하게 복원

- 로우 간 의존성 부여하여 이에 따른 차원 축소 관찰

```
[[-0.212 -0.285 -0.574 -0.44 ]
 [-0.33  1.184  1.615  0.367]
 [-0.542  0.899  1.041 -0.073]
 [-0.212 -0.285 -0.574 -0.44 ]]
```

-> 3번째 로우 = 첫 번째 로우 + 두 번째 로우,, 4번째 로우 = 첫 번째 로우로 업데이트 / 관계 up

-다시 분해

```
Output:
(4, 4) (4, ) (4, 4)
Sigma Value:
[2.663 0.807 0. 0.]
```

-> 차원 동일, Sigma 값 2개가 0으로 변화

-> 로우 벡터의 개수 2개

- 복원

=> U 행렬의 경우, Sigma와 내적 연산 수행... 따라서 Sigma 앞 2행만 추출

=> V 전치 행렬 앞 2행 추출 -> 내적하여 복원

c. Truncated SVD를 통한 행렬 분해

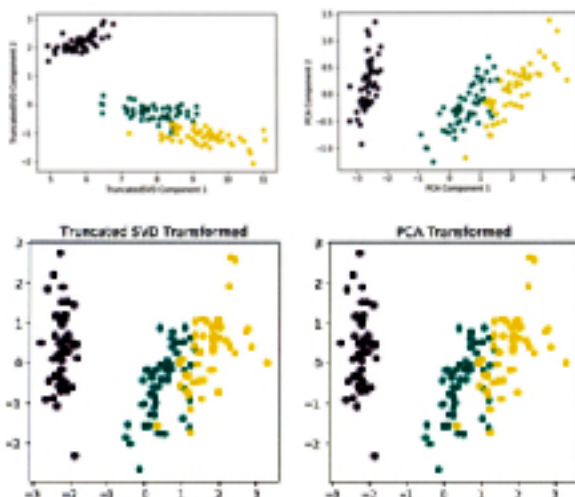
- 사이파이 scipy.sparse.linalg.svds 모듈 이용하여 분해

```
원본 행렬:
[[0.1133863 0.21876757 0.3296249 0.15194456 0.83817054 0.46791945]
 [0.5557986 0.7452394 0.2489976 0.9686194 0.95268458 0.48866885]
 [0.81927331 0.85768652 0.4493829 0.62347394 0.79537149 0.92958852]
 [0.485155 0.56738885 0.24575885 0.22573721 0.13827786 0.58898821]
 [0.82925331 0.77382256 0.9493849 0.73632338 0.17382275 0.74517176]
 [0.5181442 0.4082985 0.639515 0.82881228 0.14548493 0.81886415]]
분해 행렬: (K, 6) (6, ) (K, 6)
Sigma 값 행렬: [[3.2535887 0.88714585 0.83865238 0.55463889 0.35834626 0.8348925 ]
Truncated SVD 분해 행렬: (K, 4) (4, ) (K, 6)
Truncated SVD Sigma 값 행렬: [[0.55463889 0.83865238 0.88714585 3.2535887 ]
Truncated SVD로 분해 후 재구성 행렬:
[[0.19222945 0.21792946 0.3051823 0.54888813 0.81641485 0.42523893]
 [0.4484275 0.7284422 0.3494186 0.99148577 0.9686325 0.4754888 ]
 [0.5268662 0.88868729 0.3625735 0.59517439 0.18836734 0.93961948]
 [0.22888812 0.51826588 0.2687353 0.27388985 0.15871563 0.57156395]
 [0.8386144 0.78847467 0.9486885 0.72873231 0.1748867 0.73812389]
 [0.5826589 0.47958891 0.5453544 0.38746628 0.13735039 0.83479636]]
```

- 6 x 6 행렬 SVD 분해 시 각각 6, 6 / 6, / 6, 6 차원

- Truncated SVD 컴포넌트 = 4 일 때, 6, 4 / 4, / 4, 6 으로 각각 분해 => 또한... 완벽 X 근사 복원 값 확인 가능

d. 사이킷런 TruncatedSVD 클래스를 이용한 변환



-좌 (TruncatedSVD)와 우(PCA) 모두 클러스터링이 가능할 정도로 뛰어난 고유성을 확인 가능

-두 개의 변환 행렬 값과 원본 속성별 컴포넌트 비율값이 거의 같음을 시각화

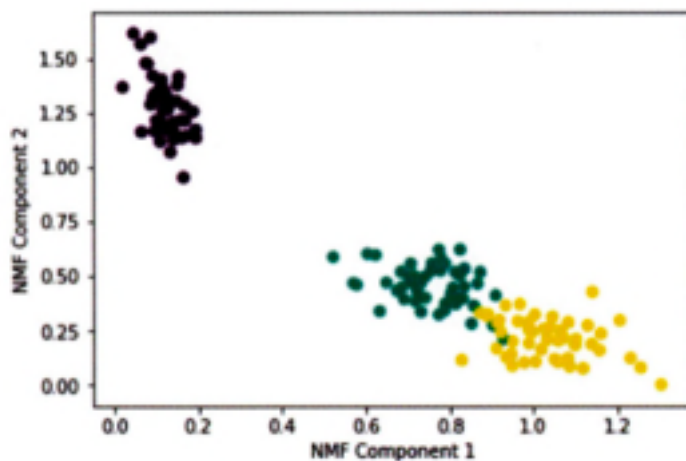
=> pca와 truncated의 차를 출력했을 때 0에 가까운 값이 도출되므로 변환이 서로 동일함

05. Non-Negative Matrix Factorization

a. 개요

- 낮은 랭크를 통한 행렬 근사 방식, 원본 행렬 내의 모든 원소 값이 모두 양수라는 것이 보장되면 조금 더 간단한 두 개의 기반 양수 행렬로 분해될 수 있는 기법
- 4×6 행렬 $V \Rightarrow 4 \times 2$ 행렬 W , 2×6 행렬 H 로 근사해 분해
- 이미지 압축을 통한 패턴 인식, 텍스트의 토픽 모델링 기법, 문서 유사도 및 클러스터링 활용
- 영화 추천과 같은 추천 영역에 활발하게 적용

b. 사이킷런 NMF 클래스 이용 시각화



- > 붓꽃 데이터 로드
- > 컴포넌트 = 2 변환
- > scatter 시각화

06. 정리

차원 축소의 목적

데이터의 주요 패턴을 효율적으로 표현하기 위해 원래의 고차원 데이터를 더 적은 수의 잠재적 요인(요소)으로 변환하는 기법.

데이터의 피쳐 수를 줄이는 것보다는 데이터의 분산을 잘 설명할 수 있는 잠재적 요인을 추출하는 데 더 큰 의미가 있음.

주요 알고리즘

PCA (Principal Component Analysis)

데이터의 변동성을 가장 잘 설명하는 축을 찾아 입력 데이터를 투영하여 차원을 축소하는 방식.

입력 데이터의 공분산 행렬을 통해 고유 벡터(Eigenvector)를 생성하여 데이터 변환.

LDA (Linear Discriminant Analysis)

PCA와 유사하지만, 클래스 간의 분산을 최대화하여 데이터를 분리하는 축을 찾는 방식.

입력 데이터의 결정 값 클래스를 최대한 분리할 수 있는 축을 선정하여 변환.

SVD (Singular Value Decomposition)

매우 많은 피쳐 데이터를 두 개의 저차원 행렬로 분리하는 행렬 분해 기법.

잠재적 요소를 효과적으로 추출하여 추천 시스템, 토픽 모델링 등에서 널리 사용됨.

NMF (Non-negative Matrix Factorization)

양수 제약 조건이 있는 행렬 분해 기법으로, 데이터의 직관적 해석에 유리함.

SVD와 마찬가지로 고차원 데이터를 두 개의 저차원 행렬로 분리.